

User Management



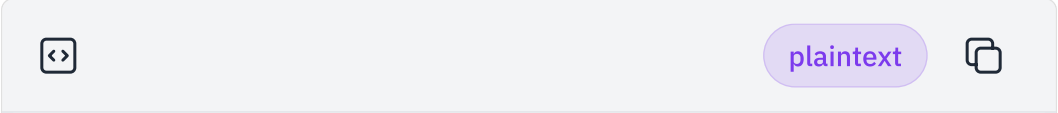
Estimated time: 1 hour and 20 mins

In this lab, you will add full user authentication capabilities to your Django–React application, including creating a superuser, configuring the frontend build, and implementing Django views for user login, logout, and registration. You will connect these backend features to the React interface, update routes and settings, and verify that users can sign up, sign in, and sign out successfully. By completing these steps, you will establish a complete user management flow within your application.

Environment Setup

If your lab workspace has been reset and you want to continue from where you left off previously, you can `git clone` or pull the latest code from your GitHub repository.

Set up the Python runtime again if the lab workspace has been reset. You can ignore this step if your lab environment is still active.



```
cd /home/project/xrwvm-fullstack_developer_capstone/server

pip install virtualenv
virtualenv djangoenv
source djangoenv/bin/activate

python3 -m pip install -U -r requirements.txt

python3 manage.py makemigrations

python3 manage.py migrate
```

Create a Superuser for your App

Let's create a superuser first.

- Stop the server if it is running.
- Create a superuser by running the command given below.

plaintext

```
python3 manage.py createsuperuser
```

With **Username**, **Email**, and **Password** entered, you should see a message saying **Superuser created successfully.**, indicating that the superuser has been created.

- Run the server by executing the command given below.

plaintext

```
python3 manage.py runserver
```

- Select the **Django Admin** button below to open the admin panel.

Django Admin

► If the page doesn't open, select here to open the URL manually.

- Log in to the admin site with the credentials you just created for the superuser.

section. You should be able to view the superuser you just created.

Build the Client-Side and Its Configuration

A starter code of the client has been provided for you.

- Open a **New Terminal** and switch to the client directory

plaintext

```
cd /home/project/xrwvm-fullstack_developer_capstone/server/
```

- Install all required packages

plaintext

```
npm install
```

- Run the command given below to build the client

plaintext

```
npm run build
```

- Open **server/djangoproj/settings.py** in the editor. Under **TEMPLATES**, you will find **DIRS**. Add **os.path.join(BASE_DIR, 'frontend/build')** to the list for the Django application to recognize the front end. It should be set up as below.

[plaintext](#)

```
'DIRS': [  
    os.path.join(BASE_DIR, 'frontend/static'),  
    os.path.join(BASE_DIR, 'frontend/build'),  
    os.path.join(BASE_DIR, 'frontend/build/static')  
],
```

- In `server/djangoproj/settings.py`, add the directories for the Django application to look for static files to the list named `STATICFILES_DIRS`. It should be set up as below.

[plaintext](#)

```
STATICFILES_DIRS = [  
    os.path.join(BASE_DIR, 'frontend/static'),  
    os.path.join(BASE_DIR, 'frontend/build'),  
    os.path.join(BASE_DIR, 'frontend/build/static'),  
]
```

Add a New Login View

Next, you need to create a new login Django view to handle a login request.

- Open `djangoapp/views.py` and uncomment the import statements in the top. Observe the login view to authenticate user. After the user logs in, it should return a JSON object with the username and status.
- Open `server/djangoapp/urls.py` and uncomment the import statements in the top.
- Configure a route for the login view by uncommenting the path entry in `server/djangoapp/urls.py`, as given below.

 python 

```
path(route='login', view=views.login_user, name='login')
```

You may refer to this lab to get more details about Django authentication:

[Django Authentication System](#)

- Configure a route for the login view by adding the path entry in `server/djangoapp/urls.py`, as given below. Login view is a REACT page rendered from a route that is configured in `/server/frontend/src/App.js`.

 python 

```
path('login/', TemplateView.as_view(template_name="inde
```

- Select the `Django App` button below to open the browser.

► If the page doesn't open, select here to open the URL manually.

- You will see the homepage. From the homepage select on the **Login** link provided. If you configured the Login view correctly, you should see the page as below.



- In the space provided, enter the username and password you gave for the superuser and login.
- The login process should go through, and you should see username and logout option displayed on the homepage, as below.



Assessment

For Option 1: AI-Graded Submission and Evaluation: Run the below command in a new terminal. Copy and paste the terminal output along with the command and save it in a text file named **loginuser** for the final project submission and evaluation.

plaintext

```
curl -X POST "<Django-URL>/djangoapp/login" \  
  -H "Content-Type: application/json" \  
  -d '{"userName": "<your-registered-username>", "password"
```

Note: Replace the Django URL, username, and password with your

For Option 2: Peer-Graded Submission and Evaluation: After you have created the login view, please take a screenshot and name it as .

`login.jpeg` or `login.png` for Peer Assignment.

Add Logout Functionality

You need to create a new logout Django view to handle the logout request.

- Open `djangoapp/views.py` and add a new logout view to handle a logout request. After the user logs out, it should return a JSON object with the username.

 plaintext 

```
logout(request) # Terminate user session
data = {"userName":""} # Return empty username
return JsonResponse(data)
```

- Configure a route for the logout view by adding a path entry in `djangoapp/urls.py`.
- In the `server/frontend/static/Home.html`, include the code given below to handle the login out of the user, in the space provided.

js

```
// Build logout URL and Make GET request to logout endpoint
let logout_url = window.location.origin+"/djangoapp/logou
const res = await fetch(logout_url, {
  method: "GET",
});

const json = await res.json();
if (json) {
  // Clear session storage and reload page
  let username = sessionStorage.getItem('username');
  sessionStorage.removeItem('username');
  window.location.href = window.location.origin;
  window.location.reload();
  // Notify user of logout
  alert("Logging out "+username+"...")
}
else {
  alert("The user could not be logged out.")
}
```

- Open a **New Terminal** and run the following command.

bash

```
cd /home/project/xrwvm-fullstack_developer_capstone/server/
npm run build
```

Run

- Select **Django Application** button below to open the browser.

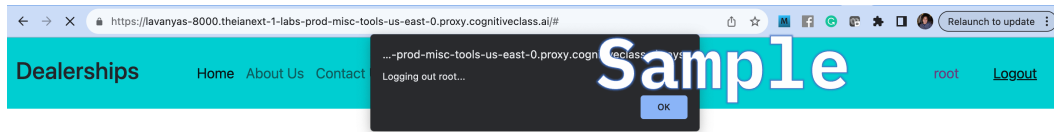
Django Application

► If the page doesn't open, select here to open the URL manually.

- If the session didn't expire, you will be logged in from the previous step. If not, login, and then, click **Logout** .

You should see an alert showing that you have successfully logged out.

Note: In case the Logout alert doesn't appear, try checking its functionality by switching to a different browser.



Assessment:

For Option 1: AI-Graded Submission and Evaluation: Run the command given below in a new terminal. Copy and paste the terminal output along with the command and save it in a text file named `logoutuser,` for the final project submission and evaluation.

plaintext

```
curl -X GET "<Django-URL>/djangoapp/logout"
```

Note: Replace the Django URL with your actual URL before running the command.

For Option 2: Peer-Graded Submission and Evaluation: After you have created the logout view, please take a screenshot and name it `logout.jpeg` or `logout.png` for Peer Assignment.

At this point, you should be able to log in and log out with the superuser you created earlier.

Add a 'Register' Functionality

You need to create a new registration Django view to handle the register request.

- Open `djangoapp/views.py` and add a new register view to handle a register request. When the user registers, a user object should be created, and the user should be logged in. It should return a JSON object with the username.

► [Click here for a suggested implementation](#)

- Configure a route for the registration view by adding a path entry in `djangoapp/urls.py`.
- Inside `frontend/src/components/Register`, create a file named `Register.jsx`. The CSS to be used for this page has already been provided.
- Add the code given below in Register.jsx.

Feel free to make changes to the look and feel.



javascript



```
import React, { useState } from "react";
import "./Register.css";
import user_icon from "../assets/person.png"
import email_icon from "../assets/email.png"
import password_icon from "../assets/password.png"
import close_icon from "../assets/close.png"

const Register = () => {
  // State variables for form inputs
  const [userName, setUserName] = useState("");
  const [password, setPassword] = useState("");
  const [email, setEmail] = useState("");
  const [firstName, setFirstName] = useState("");
  const [lastName, setlastName] = useState("");

  // Redirect to home
  const gohome = ()=> {
    window.location.href = window.location.origin;
  }

  // Handle form submission
  const register = async (e) => {
    e.preventDefault();

    let register_url = window.location.origin+"/djangoapp/r

  // Send POST request to register endpoint
    const res = await fetch(register_url, {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify({
        "userName": userName,
        "password": password,
        "firstName":firstName,
        "lastName":lastName,
        "email":email
      }),
    });

    const json = await res.json();
    if (json.status) {
      // Save username in session and reload home
      sessionStorage.setItem('username', json.userName);
    }
  }
}
```

```

    }
    else if (json.error === "Already Registered") {
        alert("The user with same username is already registe
        window.location.href = window.location.origin;
    }
};

return(
    <div className="register_container" style={{width: "50%
        <div className="header" style={{display: "flex", flexD
            <span className="text" style={{flexGrow: "1"}}>Sig
            <div style={{display: "flex", flexDirection: "row"
                <a href="/" onClick={()=>{gohome()}} style={{just
                    <img style={{width: "1cm"}} src={close_icon} alt
                </a>
            </div>
            <hr/>
        </div>

        <form onSubmit={register}>
        <div className="inputs">
            <div className="input">
                <img src={user_icon} className="img_icon" alt='
                <input type="text" name="username" placeholder
            </div>
            <div>
                <img src={user_icon} className="img_icon" alt='
                <input type="text" name="first_name" placehold
            </div>

            <div>
                <img src={user_icon} className="img_icon" alt='
                <input type="text" name="last_name" placeholde
            </div>

            <div>
                <img src={email_icon} className="img_icon" alt=
                <input type="email" name="email" placeholder="
            </div>

            <div className="input">
                <img src={password_icon} className="img_icon" a
                <input name="psw" type="password" placeholder=
            </div>

        </div>
        <div className="submit_panel">
            <input className="submit" type="submit" value="Re

```



```

    </div>
  )
}

export default Register;

```

- Configure the routes in `frontend/src/App.js`.
- In the terminal you used previously to run `npm run build`, run it again for the latest changes to be reflected.
- Open `server/djangoproj/urls.py` in the editor and add the following path to the `urlpatterns`. The routes have already been configured in `App.js`.

<>

plaintext

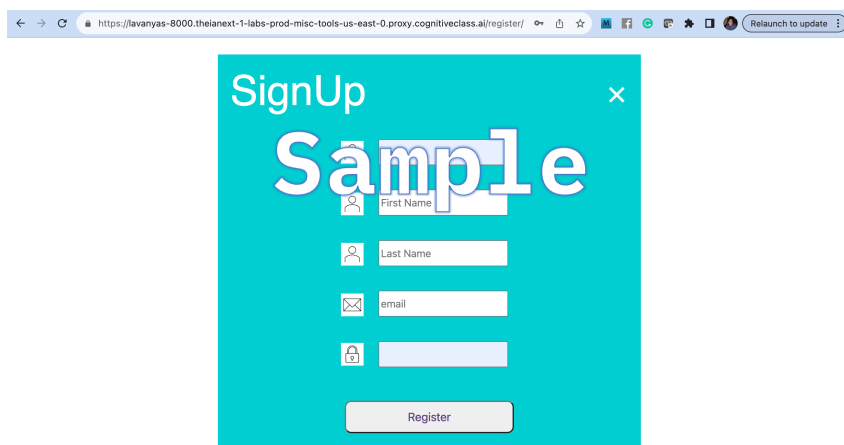
```
path('register/', TemplateView.as_view(template_name="i
```

- Select the **Django Application** button below to open the browser.

Django Application

► If the page doesn't open, select here to open the URL manually.

Logout if you are logged in and select the **Register** link. You should see a sign-up page as shown below.



- Enter user details and register. It should sign up, login, and return an active session.
- Save your changes in the GitHub repository.

Assessment:

For Option 1: AI-Graded Submission and Evaluation: Copy and paste the public Github URL of the `Register.jsx` and save it in a text file for the Final Project Submission and Evaluation.

For Option 2: Peer-Graded Submission and Evaluation: After you have created the sign-up view, please take a screenshot and name it `sign-up.jpeg` or `sign-up.png` for Peer Assignment.

Test and Submit

- Test the updated Django app with signup, login, and logout end-to-end
- Commit and push the changes to your GitHub repository

If you need to refresh your memory on how to commit and push to GitHub, please refer to this lab [Working with git](#)

- In this lab, you have added user management-related templates and views to the app.

Author(s)

[Lavanya](#)

Other Contributor(s)

Upkar Lidder

© Copyright IBM Corporation. All rights reserved.

Test and Submit

- Test the updated Django app with signup, login, and logout end-to-end

If you need to refresh your memory on how to commit and push to GitHub, please refer to this lab [Working with git](#)

- In this lab, you have added user management-related templates and views to the app.

Author(s)

[Lavanya](#)

Other Contributor(s)

Upkar Lidder

© Copyright IBM Corporation. All rights reserved.

← Add a 'Register' Functio...